

공개키 암호, 하이브리드 암호 시스템

이주희

Q&A

- Q. Rsa의 암호화, 복호화에서 암호문은 평문에 e 승을 하고 $\text{mod}N$ 을 한 결과라고 하셨는데 그럼 암호문을 다시 d 승을 한 후 $\text{mod}N$ 을 하면 평문이 다시 나오나요?
- A. 네, 맞습니다.

키 쌍	공개 키	수 e 와 수 N
	개인 키	수 d 와 수 N
암호화		$\text{암호문} = (\text{평문})^e \bmod N$ (평문을 e 제곱해서 N 으로 나눈 나머지)
복호화		$\text{평문} = (\text{암호문})^d \bmod N$ (암호문을 d 제곱해서 N 으로 나눈 나머지)

Q&A

- Q. -5의 역수가 7인 것을 설명해주셨으면 좋겠습니다.

- A. $(-5)^{-1} \bmod 12 = 7$

$$\because (-5) \bmod 12 = 7, \quad 7 \cdot 7 \bmod 12 = 49 \bmod 12 = 1$$

Q&A

- RSA 알고리즘이 특정 조건에서 가소성을 가지며, 이에 대한 대안으로 RSA-OAEP를 사용한다고 설명했습니다. 이 가소성은 구체적으로 어떤 상황에서 문제가 되며, RSA-OAEP는 이를 어떻게 해결하나요?

공개키 암호화 방법이 중간자 공격에 취약하다고 언급했습니다. 현실 세계의 어떤 상황에서 이러한 취약점이 문제가 될 수 있으며, 이를 방지하기 위해 어떤 추가적인 보안 조치를 취할 수 있나요?

양자 내성 암호화에 대해 배웠습니다. 현재 널리 사용되고 있는 암호화 방법들이 양자 컴퓨팅의 발전에 의해 위협받을 수 있는 상황에서, LWE 기반 공개키 암호화 방법이 어떻게 안전성을 제공할 수 있을까요?

NP 문제와 계산 복잡도에 대한 이해를 바탕으로, 현재 널리 사용되는 암호화 방법들이 직면할 수 있는 잠재적인 취약점은 무엇이며, 이를 어떻게 개선할 수 있나요?

중간자 공격

- 중간자(Man-in-the-middle) 공격
- 공격자 맬로리가 송신자와 수신자 사이에서 송신자에 대해서는 수신자처럼, 수신자에 대해서는 송신자처럼 행세하는 공격
- 중간자 공격을 막기 위해서는 공개키에 대한 추가적인 인증 절차가 필요함

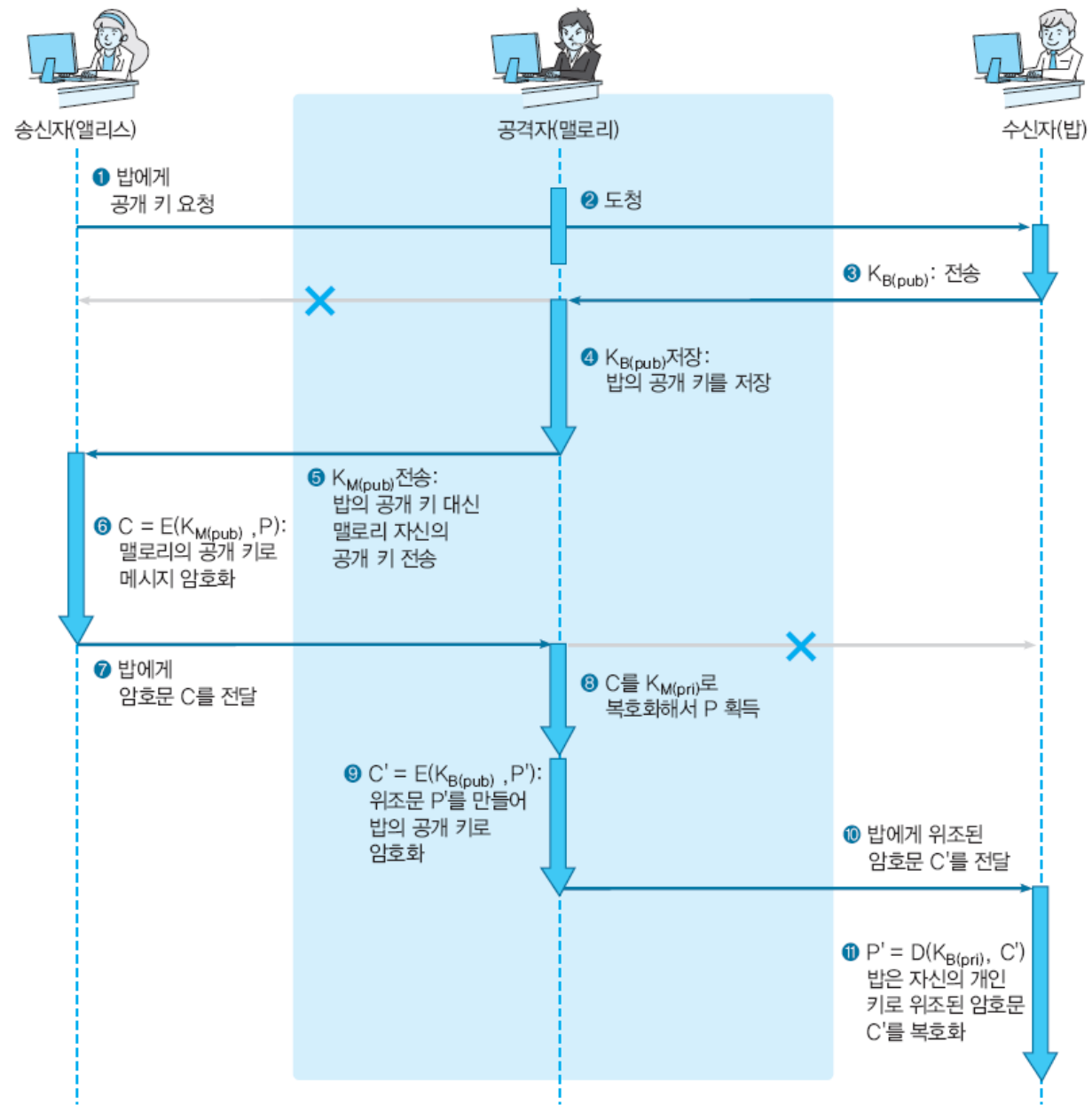
중간자 공격의 예시

- 1) 앨리스는 밥의 공개 키 요청
- 2) 맬로리는, 앨리스의 요청을 도청
- 3) 밥은 자신의 공개키($K_{B(pub)}$)를 앨리스에게 전송
- 4) 맬로리는 밥의 이메일이 앨리스에게 도달하지 못하도록 하고, 밥의 공개키를 보존
- 5) 맬로리는 자신의 공개키($K_{M(pub)}$)를 밥의 공개키라고 속여서 앨리스에게 전송
- 6) 앨리스는 자신의 메시지(P)를 밥의 공개키(실은 맬로리의 공개 키)로 암호화($C = E(K_{M(pub)}, P)$)

중간자 공격의 예시

- 7) 앨리스는 암호화한 메시지(C)를 밥에게 전송
- 8) 맬로리는 앨리스의 암호화된 메시지를 갈취해서 자신의 개인 키($K_{M(pri)}$)로 복호화($P = D(K_{M(pri)}, C)$) 하고 평문(P)을 확보
- 9) 맬로리는 앨리스 행세를 하며 위조 메일(P')을 만들고 위의 단계 (4)에서 보존해 둔 밥의 공개 키($K_{B(pub)}$)를 써서 이 위조 메일을 암호화($C' = E(K_{B(pub)}, P')$)하여 밥에게 전송
- 10) 밥은 받은 암호 메일(C')을 자신의 개인 키로 복호화하고 메일(P')을 읽게 됨

중간자 공격



Diffie-Hellman 키 교환 기법에 대한 중간자 공격의 예시

- $G = \langle g \rangle$: order $|G|$ 인 cyclic group
 - 예를 들어 소수 p 에 대해 $(\mathbb{Z}_p^*, \times)$ 는 order $(p - 1)$ 인 cyclic group
 - $g, |G|$ 는 공개된 정보



$$A = g^a$$



$$B' = g^{b'}$$

$a \in \{1, \dots, |G|\}$

$$(B')^a = g^{ab'}$$

MiTM



$a', b' \in \{1, \dots, |G|\}$

$$A^{b'} = g^{ab}$$

$$B^{a'} = g^{a'b}$$

$$A' = g^{a'}$$



$$B = g^b$$



$$(A')^b = g^{a'b}$$

Q&A

- Q. 일반적으로 p문제는 모두 NP문제라고 하셨는데 이 부분이 이해가 잘 되지 않습니다.

계산복잡도와 P vs. NP

- 다항시간 알고리즘(polynomial-time algorithm)
 - n 비트 입력과 어떤 다항식 $f(x)$ 에 대해 계산 복잡도 $O(f(n))$ 만에 계산가능한 알고리즘
 - 반대 개념은 초다항시간(superpolynomial time) 알고리즘
 - $O(g(n))$ 만에 수행 가능하고, 여기서 $g(n)$ 은 모든 다항식보다 빠르게 증가하는 어떤 함수
 - 예를 들어 지수 복잡도 $O(2^n)$ 이나 $O(n^{\log(n)})$ 등의 계산복잡도를 갖는 알고리즘이 있음
- **P 문제** : 문제를 풀기 위한 다항시간 알고리즘이 존재하는 문제들을 **P 문제**라고 함
- **NP 문제** : 인수분해 문제처럼 채점하는 다항시간 알고리즘이 존재하는 문제를 **NP 문제**라고 함
 - NP 문제는 문제를 해결하기 위해 갈 수 있는 길의 종류가 많고 어떤 길을 선택할 경우 운 좋게 다항 시간 안에 해결될 수도 있지만, 어떤 길을 선택해야 할지는 모르는 문제를 말함
 - 따라서 NP 문제들의 경우 문제를 풀기는 어렵지만, 답을 확인하기는 쉬움 (일방향성을 가짐)
 - 이러한 NP 문제는 P 문제나 채점하기 어려운 문제보다 암호를 만들기에 적합함

Q&A

- Q. ppt 45페이지에서 마지막에 $p^2 \bmod N$ 이라고 하셨는데 이것이 hash함수를 두 번 거쳤기 때문에 2승이 되는 건지 궁금합니다.
- A. ?? ($p^2 \bmod N$ 을 찾지 못함)

Q&A

- Q. 10주차 7.공개키암호(2) p.34에서 공개키에서의 N과 비밀키에서의 N은 서로 다른 값인가요? 같은 N값이라면 N이 공개키이자 비밀키인 것인데 이게 어떻게 가능한 건지 궁금합니다.
- A. 같은 N 값이 맞음.
 - 공개키 암호화 정의에서 공개키와 비밀키는 각각 암호화와 복호화의 input으로 사용되는 값들을 의미함.
 - N 값은 공개되는 값이지만, 복호화 알고리즘의 input으로 사용되므로 (N, d)를 비밀키라고 얘기함.
 - 비밀키에서 N과 같이 공개되는 값들은 생략하기도 함.

공개키 암호화

공개키 암호화(public-key encryption) 스킴 Σ 는 KeyGen, Enc, Dec 세 알고리즘들의 집합
 $\Rightarrow \Sigma = (\text{KeyGen}, \text{Enc}, \text{Dec})$

$$(sk, pk) \xleftarrow{\$} \text{KeyGen}$$

$$\text{Enc} : \mathcal{PK} \times \mathcal{M} \rightarrow \mathcal{C}$$

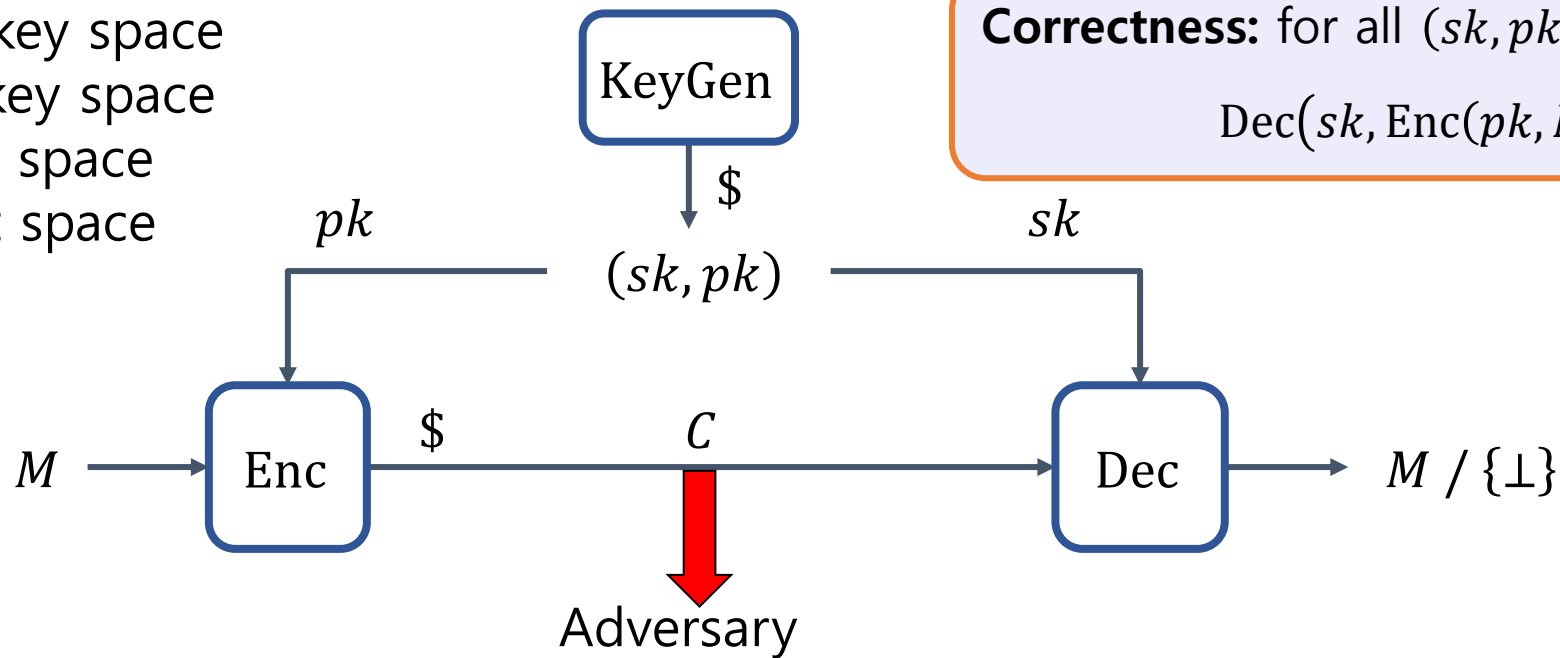
$$\text{Dec} : \mathcal{SK} \times \mathcal{C} \rightarrow \mathcal{M} \cup \{\perp\}$$

$$\text{Enc}(pk, M) = \text{Enc}_{pk}(M) = C$$

$$\text{Dec}(sk, C) = \text{Dec}_{sk}(C) = M / \perp$$

- $\mathcal{SK}$ – private key space
- $\mathcal{PK}$ – public key space
- $\mathcal{M}$ – message space
- $\mathcal{C}$ – ciphertext space

Correctness: for all $(sk, pk) \leftarrow \text{KeyGen}$:
 $\text{Dec}(sk, \text{Enc}(pk, M)) = M$



Post-Quantum Cryptography

Learning with Errors (LWE) 기반 공개키 암호

Shor's algorithm

Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*

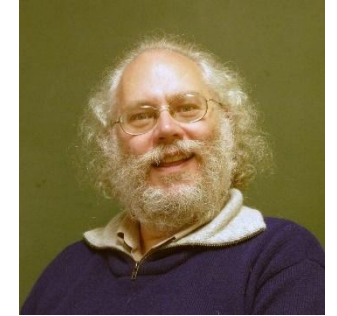
Peter W. Shor[†]

1994

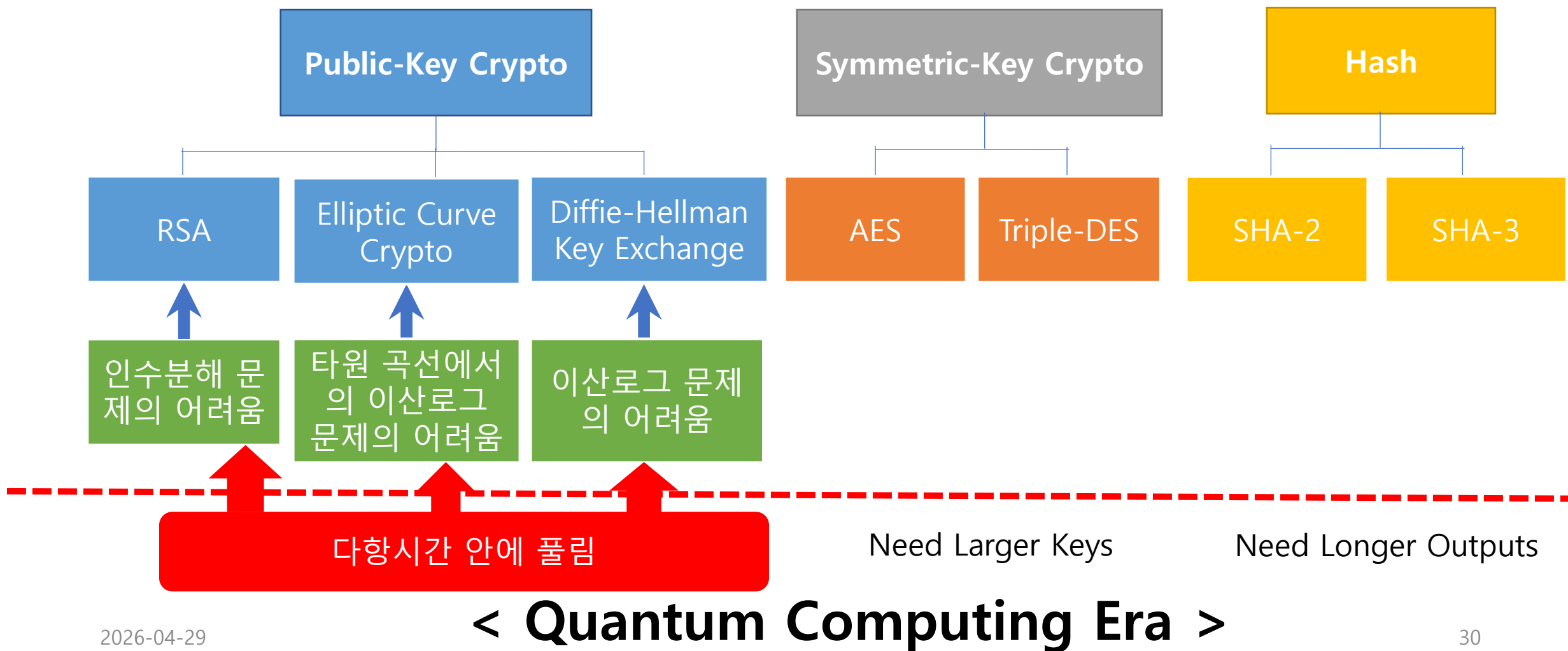
Abstract

A digital computer is generally believed to be an efficient universal computing device; that is, it is believed able to simulate any physical computing device with an increase in computation time by at most a polynomial factor. This may not be true when quantum mechanics is taken into consideration. This paper considers factoring integers and finding discrete logarithms, two problems which are generally thought to be hard on a classical computer and which have been used as the basis of several proposed cryptosystems. Efficient randomized algorithms are given for these two problems on a hypothetical quantum computer. These algorithms take a number of steps polynomial in the input size, e.g., the number of digits of the integer to be factored.

Keywords: algorithmic number theory, prime factorization, discrete logarithms, Church's thesis, quantum computers, foundations of quantum mechanics, spin systems, Fourier transforms



Post-quantum Cryptography (PQC)



Post-Quantum Cryptography (PQC)

- 단순히 기존 암호의 파라미터를 늘리는 것만으로는 새로운 양자 알고리즘의 등장에 대비할 수 없음
- 양자내성암호는 수학적 NP-Hard 난제 기반으로 양자컴퓨터로도 실질적으로 해독 불가능한 보안성 획득
- 암호키 교환 및 데이터 암호·복호화, 무결성 인증 등 다양
- 이번 시간에 다루는 Learning with Errors (LWE) 기반 공개키 암호는 PQC의 일종임

Learning with Errors 문제

Solving a linear equation system

• Q.

1	3	7
4	5	7
6	6	9
2	7	3
3	8	7
5	4	2
1	0	5
4	5	3

x_1
x_2
x_3

 $=$

7
9
2
9
6
8
2
7

(mod 10)  Find

x_1
x_2
x_3

!

; Easy!

(We can solve it by using Gaussian elimination)

Learning with errors (LWE) problem

• Q.

1	3	7
4	5	7
6	6	9
2	7	3
3	8	7
5	4	2
1	0	5
4	5	3

$\mathbb{Z}_{10}^{8 \times 3}$

x_1
x_2
x_3

+

0
2
9
1
0
1
0
8

=

7
9
2
9
6
8
2
7

(mod 10) $\rightarrow$ Find x_1, x_2, x_3 !

; Hard!

Small Error (unknown)

Decision-LWE problem

- Q. Distinguish

1	3	7	7	(mod 10)
4	5	7	1	
6	6	9	1	
2	7	3	0	
3	8	7	6	
5	4	2	0	
1	0	5	2	
4	5	3	5	

From a uniform
random sample
in $\mathbb{Z}_{10}^{8 \times 4}$

; Hard!

LWE 기반 공개키 암호

LWE 기반 공개키 암호 (Lindner, Peikert 2011)

KeyGen

$$\text{pk: } \begin{matrix} n \\ \text{m} \end{matrix} \begin{matrix} \text{A} \end{matrix}, \quad \text{b} = \begin{matrix} \text{A} \end{matrix} \begin{matrix} \text{s} \end{matrix} + \begin{matrix} \text{e} \end{matrix} \quad \text{sk: } \begin{matrix} \text{s} \end{matrix} \text{ (small)}$$

Enc(pk, M)

$$\begin{matrix} \text{r} \end{matrix} \begin{matrix} \text{A} \end{matrix} + \begin{matrix} \text{e}' \end{matrix}, \quad \begin{matrix} \text{r} \end{matrix} \begin{matrix} \text{b} \end{matrix} + \begin{matrix} \text{M} \cdot \lfloor q/2 \rfloor \end{matrix} + \begin{matrix} \text{e}'' \end{matrix}$$

- e, e', e'' : drawn from discrete Gaussian distribution
- m, n, q , 에러 분포 : LWE 문제가 충분히 어렵도록 설정



LWE 기반 공개키 암호의 복호화

- 복호화 :

$$\begin{aligned}
 & \left(\begin{array}{c} \text{orange box } r \\ \text{blue box } b \end{array} + \overset{M \cdot \lfloor q/2 \rfloor}{\text{red box}} + \text{yellow box } e'' \right) - \left(\begin{array}{c} \text{orange box } r \\ \text{blue box } A \end{array} + \text{yellow box } e' \right) \text{black box } s \\
 &= \overset{M \cdot \lfloor q/2 \rfloor}{\text{red box}} + \underbrace{(\text{yellow box } e'' - \text{yellow box } e' - \text{black box } s)}_{\text{small}}
 \end{aligned}$$

Summary

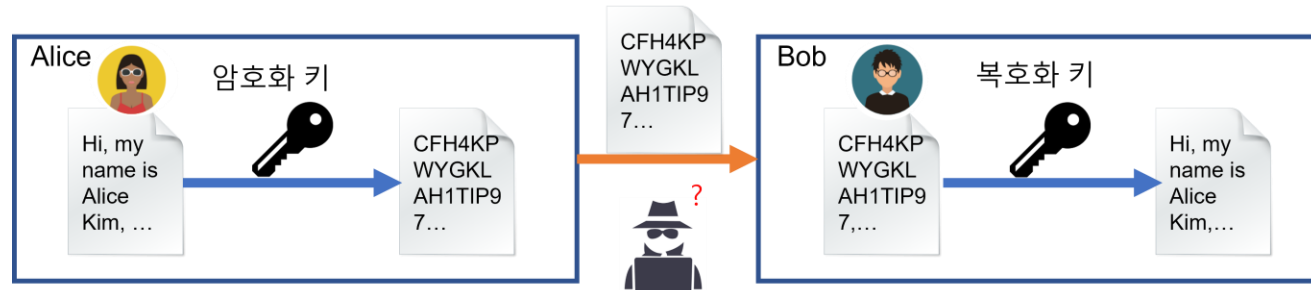
- 계산 복잡도는 알고리즘이 주어졌을 때 기본 연산(full adder, half adder 등)을 수행하는 횟수를 가리킴. 이는 input의 size에 대한 함수로 나타나며, large O notation으로 표기할 수 있음
- 인수분해 문제처럼 채점하는 다항시간 알고리즘이 존재하는 문제를 NP 문제라고 함
 - NP 문제는 암호를 만드는 데에 유용함
 - Discrete Logarithm (이산로그) 문제 역시 NP 문제
- RSA 공개키 암호화 기법은 일방향 함수 $f: x \rightarrow x^e \bmod N$ 를 사용하여 암호화하고, 이에 대한 역함수를 복호화에 사용함
- RSA의 안전성은 큰 수의 소인수분해가 충분히 어렵다는 가정에 기반함
- RSA는 가소성(malleability)을 갖고 있기 때문에 상황에 따라 안전하지 않음 → RSA-OAEP를 사용하면 이러한 문제를 해결할 수 있음
- 양자내성 암호(Post-quantum cryptography)의 일종인 LWE 기반 공개키 암호화 방법에 대해 배움

하이브리드 암호 시스템

대칭키 암호화 vs. 공개키 암호화 방식

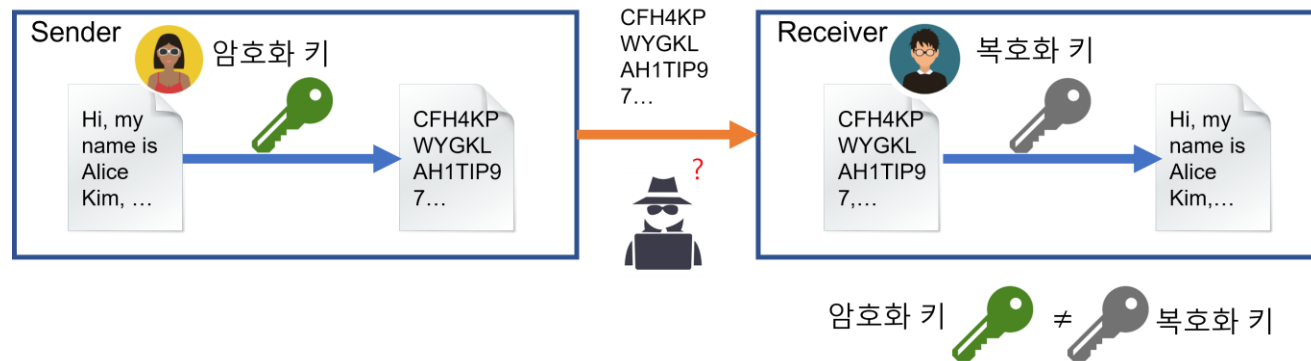
• 대칭키 암호화 방식

- 통신 기밀성 유지 가능
- 키 공유 문제 해결이 필요



• 공개키 암호화 방식

- 키 공유 문제를 해결할 수 있음



대칭키 암호화 vs. 공개키 암호화 방식

항목	대칭키 암호화 방식	공개키 암호화 방식
키의 상호관계	암호화키 = 복호화키	암호화키 $\neq$ 복호화키
암호화 키	비밀(secret key)	공개(public key)
복호화 키	비밀(secret key)	비밀(private key, secret key)
암호 알고리즘	공개	공개
대표적인 예	AES, DES	RSA, ElGamal
비밀키 공유	필요	불필요
키 개수	$n(n-1)/2$	$2n$
암호화 속도	고속	저속
경제성	높다	낮다

공개키 암호의 2가지 큰 문제

- 1) 공개키 암호는 대칭키 암호에 비해 **처리 속도가 훨씬 느림**
- 2) 공개키 암호는 중간자(man-in-the-middle) 공격에 약함
 - 하이브리드 암호 시스템 → (1)의 문제를 해결
 - 공개키에 대한 인증 → (2)의 문제를 해결
 - 인증서를 이용해서 해결할 수 있음(디지털 서명 사용)

하이브리드 암호 시스템

- **하이브리드 암호 시스템(hybrid cryptosystem)**
 - 대칭 암호와 공개 키 암호의 장점을 조합한 방법, 에니그마의 원리와 유사
 - **메시지 보호: 고속의 대칭키 암호**
 - 메시지는 대칭키 암호로 암호화하며, 이때 사용한 대칭키 암호의 비밀키를 세션 키라고 함
 - 대칭키 암호의 암호화에서 사용하는 세션 키는 의사난수 생성기로 생성
 - 대칭키 암호의 **비밀키(세션 키) 보호: 공개키 암호**
 - 메시지의 길이에 비해 일반적으로 사용되는 대칭키 암호의 비밀키 길이는 상대적으로 짧음(128~256-bit 등). 이처럼 짧은 데이터를 암호화 하기 때문에 공개키 암호의 속도가 느리더라도 문제가 되지 않음

하이브리드 암호 시스템

- 하이브리드 암호 시스템에 사용되는 3 가지 암호 기술
 - 의사난수 생성기
 - 대칭키 암호 (E,D)
 - 공개키 암호 (Enc, Dec)

의사난수 생성기

- Pseudo Random Number Generator (PRNG)
 - 생각 외로 컴퓨터는 난수를 간단히 만들 수 없음
 - 처음에 주어지는 초기값(시드)을 이용하여 이미 결정되어 있는 메커니즘을 통해 난수를 생성함
 - 초기값(시드)을 아는 공격자는 의사 난수열을 알게 됨, 시드는 비밀값
 - 임의성(Randomness), 비예측성(Unpredictability), 시드 요구사항을 만족해야 함
 - 임의성 : PRNG에 의해 생성된 비트 스트림은 랜덤하게 보여야 함
 - 비예측성 : 이전 비트들에 대한 정보가 있어도 다음 출력 비트를 예측할 수 없어야 함, 생성된 어떤 값의 정보를 통해서도 시드를 결정할 수 없어야 함
 - 시드 요구사항 : 시드는 예측 불가능해야 함. True Random Number Generator (TRNG)로 생성된 난수 사용 권고(SP800-90, [SP 800-90A Rev. 1, Random Number Generation Using Deterministic RBGs | CSRC](#))
- 세션 키 생성에 PRNG 사용
 - PRNG의 품질이 나쁘면 만들어지는 세션 키를 공격자가 추측할 위험성이 있음
 - 세션 키 중 일부 비트라도 추측할 수 없도록 주의

의사난수 생성기 예시 (1)

- Blum Blum Shub 생성기
 - 개발자들의 이름에서 유래 [BLUM86]
 - 어떠한 특정 목적의 알고리즘에서 암호학적 강도를 증명하기 위해 가장 강력하게 통용되는 수단
 - 암호학적으로 안전한 의사난수 비트 생성기로 불림
 - CSPRBG : Cryptographically Secure Pseudo Random Bit Generator
 - 속도는 느림
 - 소인수분해의 어려움에 기반
- 세팅 :
 - $p \bmod 4 = q \bmod 4 = 3$ 인 소수 p, q 선택
 - $n = p \cdot q$
 - n 과 서로소인 s (시드) 선택

$$X_0 = s^2 \bmod n$$

$$B_0 = X_0 \bmod 2$$

for $i = 1, 2, \dots,$

$$X_i = X_{i-1}^2 \bmod n$$

$$B_i = X_i \bmod 2$$

난수 생성 : $B_0, B_1, B_2, \dots$

의사난수 생성기 예시 (1)

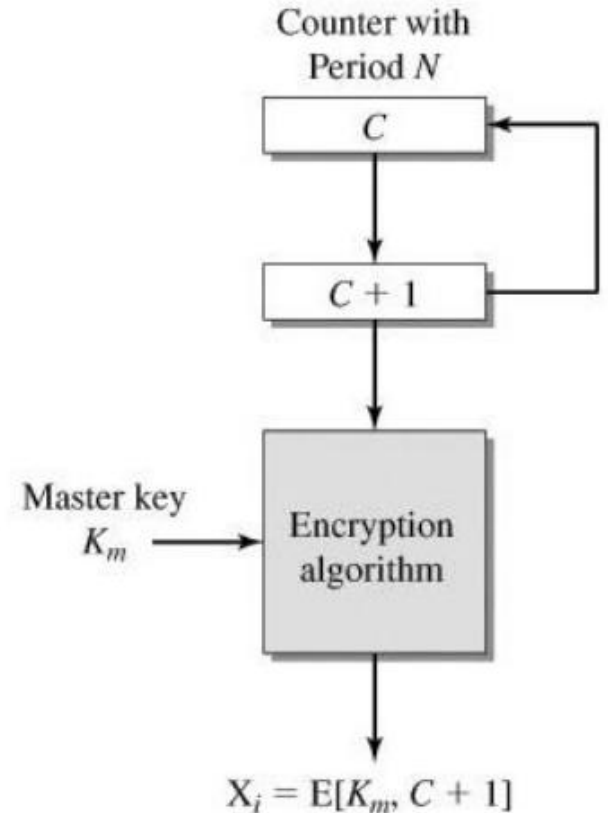
- BBS 생성기의 연산 예
 - $n = 192648 = 383 \times 503$
 - seed $s = 101355$

i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

I	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

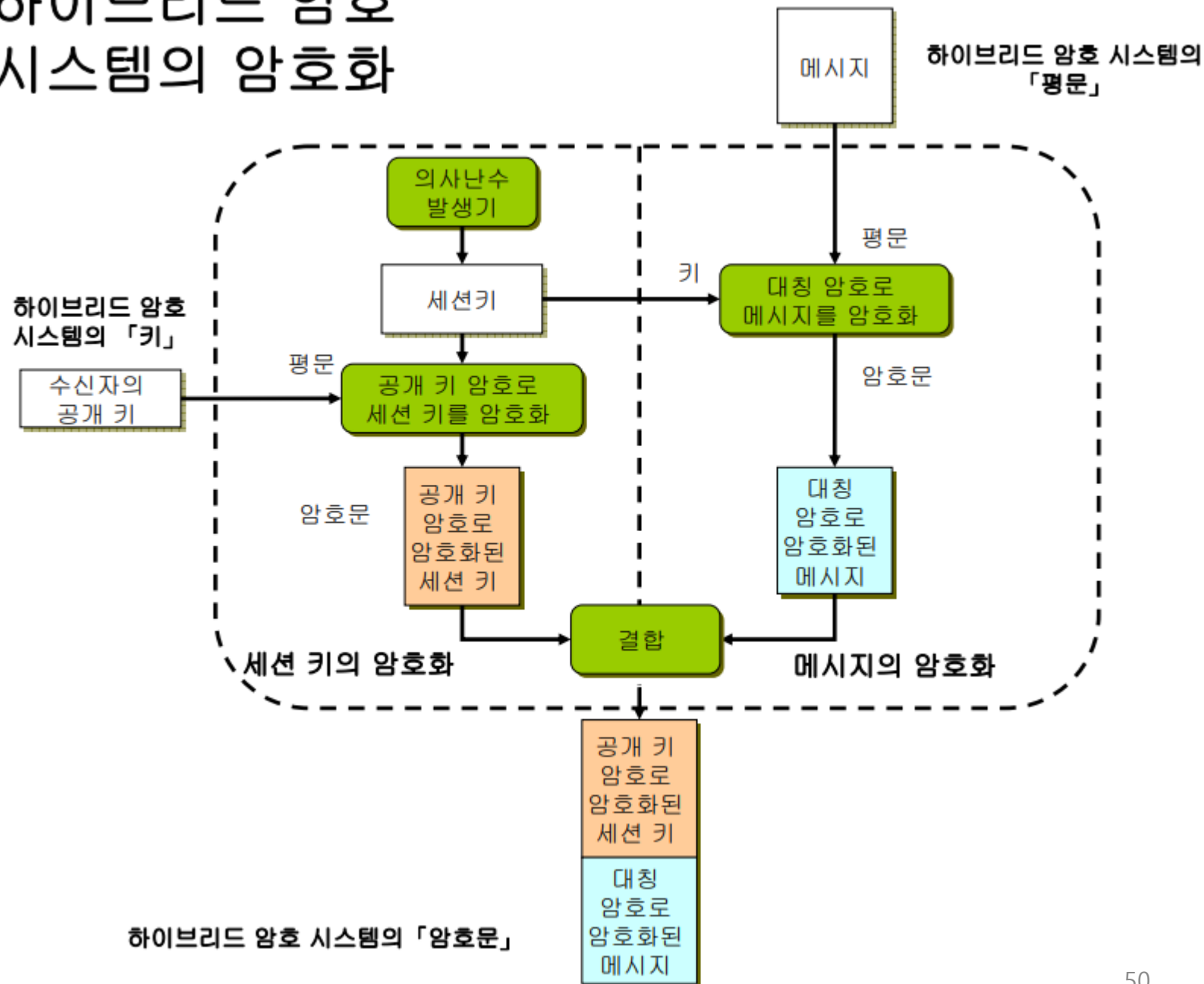
의사난수 생성기 예시 (2)

- 블록암호를 의사난수 생성기로 이용할 수 있음
- 카운터 모드 :
 - $X_i = E_{K_m}[i]$
 - 마스터키 K_m 를 시드로 하여 세션 키를 생성하기 위한 방법
 - 카운터 i 가 블록암호의 입력값으로 사용됨
 - 카운터를 1씩 증가시키면서 X_i 를 생성하고 사용함
 - 만약 사용되는 블록암호 알고리즘이 DES라면 카운터는 2^{56} 주기로 사용됨



하이브리드 암호 시스템 - 암호화

하이브리드 암호 시스템의 암호화

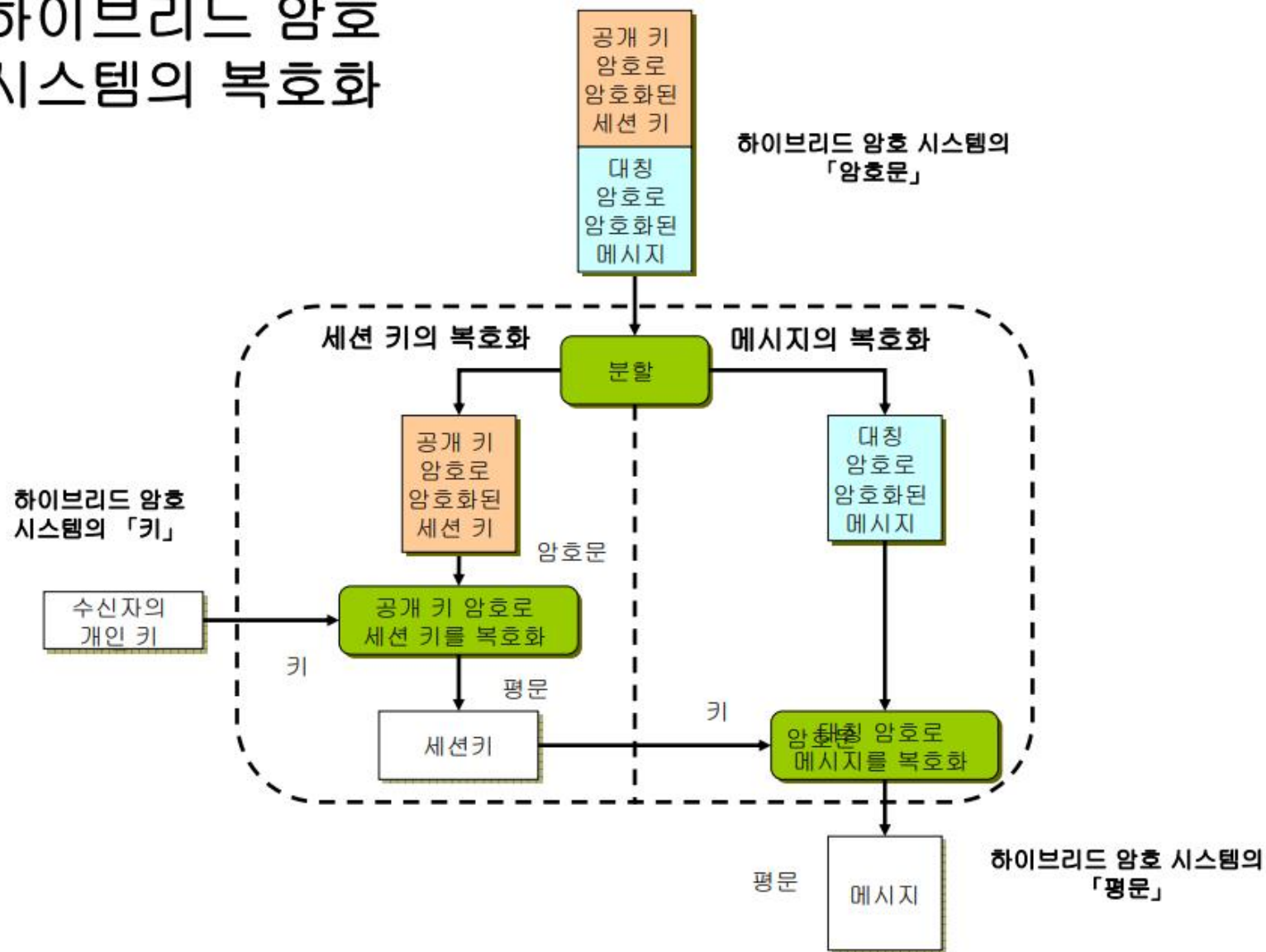


하이브리드 암호 시스템 - 암호화

- P: 평문
- K_{pub} : 수신자의 공개 키
- 세션 키 암호화
 - $C_2 = \text{Enc}(K_{pub}, K)$
 - C_2 : 공개키 암호(K_{pub})로 암호화된 세션 키
- 메시지 암호화
 - $C_1 = E(K, P)$
 - C_1 : 대칭키 암호로 암호화된 메시지
- 결합
 - $C = (C_1, C_2)$: 암호문

하이브리드 암호 시스템 - 복호화

하이브리드 암호 시스템의 복호화



하이브리드 암호 시스템 - 복호화

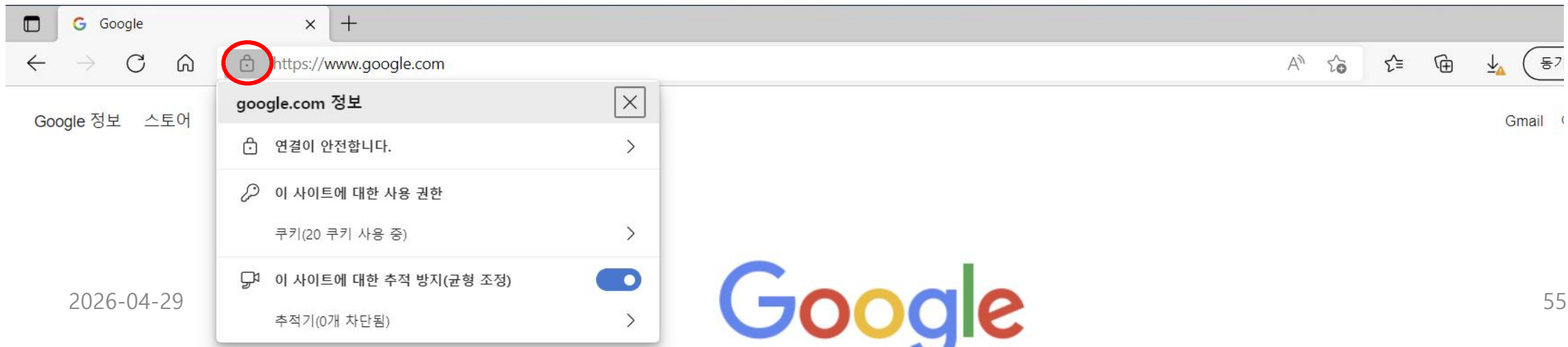
- 암호문 분할 : $C = (C_1, C_2)$
 - $C_1 = E(K, P)$: 대칭키 암호로 암호화된 암호문
 - $C_2 = \text{Enc}(K_{\text{pub}}, K)$: 수신자의 공개키(K_{pub})로 암호화된 세션 키
- $C_2 = \text{Enc}(K_{\text{pub}}, K)$ 복호화
 - 수신자의 개인 키(K_{pri})가 필요
 - 개인 키를 가지고 있는 사람이 아니면 세션 키를 복호화 할 수 없음
 - $K = \text{Dec}(K_{\text{pri}}, C_2)$: 수신자의 개인키로 복호화된 세션 키는 메시지 복호화 키로 이용
- 메시지 복호화
 - $P = D(K, C_1)$

하이브리드 암호 시스템의 구체적인 예시

- TLS(SSL) : Web에서의 암호화 통신에서 사용되는 프로토콜
 1. 지원 가능한 알고리즘 서로 교환
 2. 키 교환 및 인증
 3. 대칭키 암호로 암호화하여 데이터 통신
- PGP : 이메일 내용 자체를 암호화하여 보내는 사람과 받는 사람만이 원문을 알 수 있도록 하는 암호화 소프트웨어
- TLS, PGP에서는 하이브리드 암호 시스템에 추가해서 디지털 서명, 비밀키의 관리에 대한 처리도 연관됨

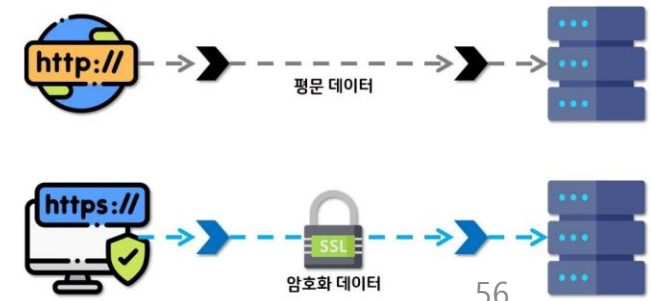
Transport Layer Security (TLS)

- HTTPS의 'S' : 통신 기밀성을 지키면서, 중간자 공격(Man-in-the-Middle Attack)을 막기 위한 프로토콜
- 전송 계층 보안(Transport Layer Security, TLS)
 - 예전에는 보안 소켓 계층 (Secure Socket Layer, SSL)이라고 불렸음
 - 서버와 클라이언트 사이의 연결을 보호함
 - 웹사이트와 방문자 사이의 연결, 이메일 서버와 클라이언트, 모바일 응용 프로그램과 그 서버 등 대부분의 보안 연결을 보호함



Transport Layer Security (TLS)

- 1995년 넷스케이프(Netscape) 브라우저의 개발사인 넷스케이프사가 SSL (SSL 2.0)을 공개함 : 보안취약점이 발견되어 1996년 SSL 3.0으로 업그레이드됨
- 1999년도에 SSL 3.0에 대한 업그레이드 버전으로 TLS 1.0 버전을 공개, 2006년 TLS 1.1로 업그레이드
 - 현재 TLS 1.0과 TLS 1.1은 지원 중단됨
- 2008년 TLS 1.2로 업그레이드
 - 취약한 해시함수 알고리즘을 변경(SHA1 → SHA2)
 - TLS 1.2는 CBC 모드의 AES를 지원하는데, 이 조합은 패딩에 대한 공격에 취약
- 2018년 TLS 1.3으로 업그레이드함
 - 서버에서 인증서를 암호화하여 전달하도록 개선
 - 최초 연결시 암호화 통신 개시 절차를 간소화하여 성능을 향상 시킴
 - 그 외에도 오래된 암호화 기술 등 폐기



하이브리드 암호 시스템의 안전성

강한 하이브리드 암호 시스템

- 하이브리드 암호 시스템이 의사난수 생성기, 대칭키 암호, 공개 키 암호를 조합시킨 것이기 때문에, 하이브리드 암호 시스템이 강한 보안성을 갖기 위해 각각의 기술 요소가 강한 보안성을 가져야 함
- 또한 강도의 밸런스가 중요함

강한 하이브리드 암호 시스템

- 의사난수 생성기
 - 하이브리드 암호 시스템에서는 세션 키를 만드는 데 의사난수 생성기를 사용함
 - 의사난수 생성기의 품질이 나쁘면 만들어지는 세션 키를 공격자가 추측할 위험성이 있음
 - 의사난수 생성기의 품질이 나쁘다는 의미는 생성되는 의사난수에 대해 임의성이나 비예측성 등이 약한 경우를 의미함

강한 하이브리드 암호 시스템

- 대칭키 암호화

- 하이브리드 암호 시스템에서는 메시지를 암호화할 때 대칭키 암호를 사용
- 강한 대칭키 암호 알고리즘을 사용하고, 키 길이도 충분히 길게 할 필요 있음
- 적절한 블록암호 운영모드를 사용할 필요 있음

- 공개키 암호화

- 하이브리드 암호 시스템에서는 세션 키를 암호화할 때 공개키 암호를 사용
- 강한 공개키 암호 알고리즘을 사용하고, 키 길이도 충분히 길게 할 필요 있음

강한 하이브리드 암호 시스템

- 키 길이의 밸런스
 - 하이브리드 암호 시스템에서는 대칭키 암호와 공개키 암호 두 가지가 모두 사용됨
 - 어느 한 쪽의 안전성이 취약한 경우, 공격이 그쪽으로 집중될 수 있음
 - 따라서 대칭키 암호화, 공개키 암호화 방식에서의 키 길이는 양쪽이 같은 안전성을 갖도록 밸런스를 맞추는 것이 바람직함

Summary

- 하이브리드 암호 시스템에서는 대칭키 암호와 공개키 암호를 조합하여 양쪽의 장점을 살리는 시스템을 구축함
- 하이브리드 암호 시스템에서 메시지는 대칭키 암호화로, 세션키는 공개키 암호화를 사용하여 암호화됨
- 암호 기술을 조합해서 보다 유용한 시스템을 구축하는 것은 자주 사용되는 기법
 - 예시: 3DES, DES를 3번 조합해서 DES보다 안전한 대칭키 암호를 만듦

[별첨] Post-Quantum Cryptography (PQC)

- **Top candidates:**
 - Lattice-based cryptography
 - Code-based cryptography
 - Multivariate cryptography
 - Hash-based cryptography
 - Isogeny-based cryptography